

# JAVASCRIPT, IN TABLES

The lecture taught what bites. This is the part you look up — keep it open while you write, and stop guessing.

► **THE NINE DEMOS ARE YOURS TOO**, in  `demos/` . Everything in this deck can be checked against one of them.

# THE EIGHT FALSY VALUES. THERE ARE NO OTHERS.

| VALUE                  | NOTE                                             |
|------------------------|--------------------------------------------------|
| <code>false</code>     | the obvious one                                  |
| <code>0, -0, 0n</code> | every kind of zero                               |
| <code>""</code>        | <b>the empty string.</b> An untouched form field |
| <code>null</code>      | your absence                                     |
| <code>undefined</code> | the language's absence                           |
| <code>NaN</code>       | a number that is not one                         |

`"0"`, `"false"`, `[]` and `{}` are all truthy. Three of those four arrive from a form or from JSON.

## **==**, IN THE ORDER THE RULES ARE APPLIED

- 1 Same type**  
compare directly. No conversion.
- 2 null and undefined**  
equal to each other only — so `value == null` is the one exception worth writing.
- 3 Object vs primitive**  
`toString()` it, then start over.
- 4 Boolean vs anything**  
the boolean becomes `0` or `1`.
- 5 String vs number**  
the **string** becomes a number.

► **HENCE** `'' == 0` **TRUE** AND `'' == '0'` **FALSE**. Not transitive — `demos/01` computes the whole table.

# ARRAY METHODS — WHAT EACH RETURNS

| METHOD                                                                                                 | RETURNS                             | TOUCHES THE ORIGINAL? |
|--------------------------------------------------------------------------------------------------------|-------------------------------------|-----------------------|
| <code>map</code>                                                                                       | a new array, same length            | no                    |
| <code>filter</code>                                                                                    | a new array, fewer items            | no                    |
| <code>find</code>                                                                                      | one item, or <code>undefined</code> | no                    |
| <code>findIndex</code>                                                                                 | an index, or <code>-1</code>        | no                    |
| <code>some</code> · <code>every</code>                                                                 | a boolean                           | no                    |
| <code>reduce</code>                                                                                    | one value, any shape                | no                    |
| <code>slice</code>                                                                                     | a new array                         | no                    |
| <code>sort</code> · <code>reverse</code>                                                               | <b>the same array</b>               | <b>YES</b>            |
| <code>push</code> · <code>pop</code> · <code>shift</code> · <code>unshift</code> · <code>splice</code> | varies                              | <b>YES</b>            |

The two dangerous ones are dangerous *because* they also return the array, so they look like the safe group.

# COMPARATORS

## SORTING

## WRITE

numbers, ascending

```
(a, b) => a - b
```

numbers, descending

```
(a, b) => b - a
```

text

```
(a, b) => a.localeCompare(b, 'he')
```

a field

```
(a, b) => a.year - b.year
```

missing values last

```
(a.year ?? Infinity) - (b.year ?? Infinity)
```

with a tie-breaker

```
b.rating - a.rating || a.title.localeCompare(b.title, 'he')
```

## NEVER

```
list.sort()  
list.sort((a, b) => a.title > b.title)
```

The first compares strings. The second returns a boolean, and `sort` wants a number.

## ALWAYS ON A COPY

```
[...list].sort(...)
```

## var, let, const

|                             | VAR                               | LET                       | CONST                     |
|-----------------------------|-----------------------------------|---------------------------|---------------------------|
| Scope                       | function                          | block                     | block                     |
| Hoisted                     | yes, as <code>undefined</code>    | <b>yes, uninitialised</b> | <b>yes, uninitialised</b> |
| Read before declaration     | <code>undefined</code> , silently | throws                    | throws                    |
| Reassign the name           | yes                               | yes                       | no                        |
| Mutate the value            | yes                               | yes                       | <b>yes</b>                |
| Redeclare in the same scope | yes                               | no                        | no                        |

`let` **is** hoisted. It is simply not initialised, and that gap is the temporal dead zone.

# this, BY CALL SITE

## WRITTEN AS

## THIS IS

`new Thing()`

the new object (not used in this course)

`fn.call(x) · fn.apply(x) · fn.bind(x)`

`owner.fn()`

`owner` — the thing before the dot

`fn()`

`undefined` in a module

**an arrow function**

**none of the above.** Whatever `this` was where it was written

### THE BUG

```
button.addEventListener(  
  'click', shelf.render)
```

Passed, not called through the dot. The object is gone.

### THE FIX

```
button.addEventListener(  
  'click', () => shelf.render())
```

**Keep the dot.** Not "use an arrow".

# ARROW FUNCTIONS

| WRITTEN                                  | RETURNS                                    |
|------------------------------------------|--------------------------------------------|
| <code>(x) =&gt; x * 2</code>             | <code>x * 2</code>                         |
| <code>(x) =&gt; { x * 2 }</code>         | <code>undefined</code> — braces are a body |
| <code>(x) =&gt; { return x * 2; }</code> | <code>x * 2</code>                         |
| <code>(x) =&gt; ({ id: x })</code>       | an object — note the parentheses           |
| <code>() =&gt; ...</code>                | no parameters                              |
| <code>(a, b) =&gt; ...</code>            | two parameters                             |

Arrow for a callback. **function** for a method. That is the whole decision.

# DESTRUCTURING, SPREAD, REST

```
1  const { title, year = 'unknown' } = album;           // with a default
2  const { album: { title: name } } = entry;             // nested, renamed
3  const [first, ...rest] = items;                       // array, with rest
4
5  const next = { ...state, query: 'kid a' };            // copy, then override
6  const grown = [...items, newItem];                   // append without push
7  const merged = { ...a, ...b };                       // b wins on collisions
8
9  function f(a, ...others) { }                         // rest parameter
10 function g({ retries = 1 } = {}) { }                 // destructured parameter
```

## ORDER DECIDES

`{ ...state, query }` and `{ query, ...state }` are different. The last one wins.

## BOTH ARE SHALLOW

A nested object inside the copy is still the **same** object.

# ? . AND ??

| WRITTEN             | MEANS                                                                                                  |
|---------------------|--------------------------------------------------------------------------------------------------------|
| <code>a?.b</code>   | <code>undefined</code> if <code>a</code> is null/undefined, instead of throwing                        |
| <code>a?.[i]</code> | the same, for an index                                                                                 |
| <code>a?.()</code>  | call it only if it is a function                                                                       |
| <code>a ?? b</code> | <code>b</code> only when <code>a</code> is <b>null or undefined</b>                                    |
| <code>a    b</code> | <code>b</code> whenever <code>a</code> is <b>falsy</b> — including <code>0</code> and <code>' '</code> |

A rating of `0` is a rating and a price of `0` is free. `||` throws both away.



# MODULES

```
1 // state.js
2 export const getState = () => state;
3 export function selectVisible(state) { ... }
4
5 // app.js
6 import { getState, selectVisible } from './state.js';
7 import { getState as read } from './state.js'; // renamed
```

## 1 The .js is required

it is a URL, not a module name.

## 2 Relative paths need ./

and modules are strict, so `this` is `undefined` in a plain call.

## 3 `type="module"` is deferred

the DOM exists; no `DOMContentLoaded`.

## 4 Not over `file://`

Live Server, always.

# THE EVENT LOOP

- 1 Run**  
all synchronous code, to the end. Nothing interrupts it.
- 2 Drain microtasks**  
`.then`, `await`, `queueMicrotask` — **all of them**, including ones added while draining.
- 3 One task**  
a timer, a click, a network response.
- 4 Back to 2**  
microtasks again, before the next task.

## MICROTASKS

Promises · `await` ·  
`queueMicrotask` ·  
`MutationObserver`

## TASKS

`setTimeout` · `setInterval` ·  
events · network callbacks

## THE RULE

**Neither queue is consulted while anything is on the stack.**

# CONVERTING ON PURPOSE

| WRITTEN                           | RESULT           | WATCH                                  |
|-----------------------------------|------------------|----------------------------------------|
| <code>Number('42')</code>         | 42               |                                        |
| <code>Number('')</code>           | 0                | the empty field                        |
| <code>Number('42kg')</code>       | NaN              | never throws                           |
| <code>parseInt('42kg', 10)</code> | 42               | always pass the base                   |
| <code>String(42)</code>           | '42'             |                                        |
| <code>Boolean('false')</code>     | true             | a non-empty string                     |
| <code>Number.isNaN(x)</code>      | the test for NaN | <code>x === NaN</code> is always false |

# THE FEATURE\_SPEC TEST

## 1 Testable

could somebody else tell whether it happened, without asking you?

## 2 Bounded

one feature, one file. If the sentence has "and", it is two.

## 3 Edged

empty · missing · duplicated · very long · none at all.

## 4 Worded

the text that appears on screen, not "a suitable message".

### NOT A SPEC

"Filter the list." · "Handle errors properly." · "Show a suitable message."

### A SPEC

"An empty query shows everything." · "With no matches the list is empty and the counter reads 0."

### \$7

Filled in **after** the code exists. It is the part that is graded.

# THE FIVE THAT COST MARKS

## WRITTEN

```
if (value == 0)
```

```
list.sort()
```

```
function f(list) { list.sort(...) }
```

```
el.on('click', obj.method)
```

```
(x) => { x * 2 }
```

## WHAT ACTUALLY HAPPENS

' ' and [] pass too

compares strings, and reorders **your** array

the caller's array is reordered

**this** is lost

returns **undefined**, silently

► **EVERY ONE OF THESE HAS A DEMO.** [demos/01](#), [07](#), [03](#), [06](#) and the deck's act 5.

# WHERE TO LOOK, IN ORDER

- 1 The console**  
read the message. It names the line and the value.
- 2 The demos**  
`demos/` — nine pages that measure what they claim.
- 3 This deck**  
the tables above.
- 4 MDN**  
search "mdn" plus the method name. Not a tutorial site.
- 5 The primer**  
`primer/` , if the syntax itself is the problem.
- 6 In week 13, none of it**  
which is what the exercises are for.